

STUDIEGIDS - MOBILE APP (React Native + Expo + Firebase)

Deze gids is gemaakt zodat jij de hele app kan uitleggen aan je docent, ook als je nog beginner bent in React.

1. Wat voor project is dit?

Dit project is een mobiele voetbal-app gebouwd met:

- React Native (UI voor iOS/Android)
- Expo (tooling, bundler, runtime)
- Expo Router (navigatie op basis van mappen en bestanden)
- Firebase Auth (inloggen/registeren)
- Firestore (database)

De app laat gebruikers:

- account maken / inloggen
- wedstrijden bekijken
- zelf een wedstrijd aanmaken
- een veld boeken
- details van een wedstrijd bekijken en chatten
- profiel en eigen data bekijken

2. Projectstructuur (hoog niveau)

- app/: alle schermen/routes (Expo Router)
- config/: centrale config, hier Firebase setup
- scripts/: utility scripts (zoals seeding)
- assets/: afbeeldingen/iconen/splash
- secrets/: gevoelige credentials voor lokaal gebruik
- root config files: app.json, package.json, tsconfig.json, babel.config.js, etc
- .

3. Runtime flow: van app-start naar scherm

1. index.ts laadt expo-router entry.
2. Expo Router leest de map app/ en bouwt routes.
3. app/_layout.tsx definieert root Stack-navigatie.
4. app/index.tsx is login/register scherm.
5. Na login gaat app naar /(tabs)/matches.
6. app/(tabs)/_layout.tsx tekent bottom tab bar.

4. Uitleg per bestand

4.1 Root configuratie

app.json

Waarom bestaat dit bestand:

- Expo app-config: naam, iconen, splash, platform-opties, plugins.

Belangrijkste keys:

- name, slug, version: app-identiteit
- scheme: deep-link URI schema
- icon, splash, adaptiveIcon, favicon: branding
- plugins: ["expo-router"] activeert router plugin
- android.edgeToEdgeEnabled: moderne fullscreen rendering

Wat je kan zeggen aan docent:

- "app.json is de centrale Expo manifest/config voor build- en runtime-instellingen."

package.json

Waarom:

- dependencies, scripts en project metadata.

Belangrijke scripts:

- start: expo start
- android / ios / web: platform launch
- seed:firebase: Firestore data vullen

Belangrijke dependencies:

- expo-router: bestandsgebaseerde navigatie
- firebase + firebase-admin: client + admin workflows
- nativewind + tailwindcss: utility styling support

Wat je kan zeggen:

- "package.json bepaalt welke libraries de app gebruikt en welke npm scripts beschikbaar zijn."

tsconfig.json

Waarom:

- TypeScript compiler settings.

Inhoud:

- extends expo/tsconfig.base.json
- strict true

Waarom belangrijk:

- strict mode vangt typefouten vroeg.
- juiste Expo tsconfig voorkomt JSX/type issues.

babel.config.js

Waarom:

- Babel transpile config voor Expo bundling.

Inhoud:

- preset babel-preset-expo.

tailwind.config.js

Waarom:

- NativeWind/Tailwind scan-locaties en thema settings.

Inhoud:

- content paden voor app/ en components/.
- theme.extend en plugins placeholders.

index.ts

Waarom:

- echte entriypoint die Expo registreert.

Wat doet het:

- importeert expo-router/entry.
- router neemt app root over.

nativewind-env.d.ts

Waarom:

- TypeScript type reference voor NativeWind.

App.tsx.old

Waarom:

- oude boilerplate die waarschijnlijk vervangen is door expo-router flow.

Belangrijk:

- niet actief als entry (main staat op expo-router/entry).

.gitignore

Waarom:

- voorkomt committen van dependencies, build output, lokale env files, secrets.

Veiligheidsrelevant:

- firebase-admin*.json en serviceAccountKey*.json zijn genegeerd.

4.2 Firebase en scripts

config/firebase.ts

Waarom:

- centrale initialisatie van Firebase client SDK.

Wat gebeurt er:

- initializeApp(firebaseConfig)
- export auth = getAuth(app)
- export db = getFirestore(app)

Gebruik:

- elk scherm importeert auth/db uit 1 plaats.

Docent-antwoord:

- "Deze file voorkomt duplicate init en is de single source of truth voor Firebase client services."

scripts/seedFirestore.js

Waarom:

- script om basisdata te vullen in Firestore (clubs + appConfig).

Wat doet het technisch:

- laadt firebase-admin veilig (met foutmelding als ontbreekt)
- leest service account uit env vars
- initialiseert Admin SDK
- schrijft clubs in collection clubs
- schrijft settings in appConfig/settings

Belangrijke env vars:

- FIREBASE_SERVICE_ACCOUNT_KEY_PATH
- FIREBASE_SERVICE_ACCOUNT_KEY_JSON
- optional FIREBASE_PROJECT_ID

Docent-antwoord:

- "Seeding gebeurt via Admin SDK omdat client writes vaak door Firestore rules geblokkeerd worden."

secrets/firebase-admin.json

Waarom:

- service account sleutel voor lokale admin scripts.

Belangrijk:

- nooit publiek delen.
- bij voorkeur niet committen.

4.3 Assets

assets/icon.png

- app icoon

assets/adaptive-icon.png

- Android adaptive icon foreground

assets/splash-icon.png

- splash screen afbeelding

assets/favicon.png

- web favicon

4.4 Routes en schermen (app/)

app/_layout.tsx

Waarom:

- root Stack navigator configuratie.

Routes in Stack:

- index (login)
- (tabs) (tab-based hoofdapp)
- match/[id] (detail pagina)

Belangrijk concept:

- layout-bestanden in Expo Router werken als navigatie-skeleton.

app/index.tsx (Login/Register)

Waarom:

- startscherm voor authenticatie.

State variabelen:

- email, password, name
- isLogin (mode toggle)
- loading

Belangrijke logic:

- isLogin true: signInWithEmailAndPassword
- isLogin false: createUserWithEmailAndPassword + users doc maken
- daarna router.replace('/(tabs)/matches')

Docent-vraag voorbeeld:

- "Waarom setDoc bij register?"

Antwoord:

- "Omdat Auth alleen identity maakt; app-specifieke data zoals level en naam bewaren we in Firestore users-collectie."

app/(tabs)/_layout.tsx

Waarom:

- bottom tab navigatie en styling.

Tabs:

- matches
- create
- book
- profile

Belangrijk:

- headerShown false omdat elk scherm eigen custom header tekent.

app/(tabs)/matches.tsx (Wedstrijden overzicht)

Doel:

- alle actieve wedstrijden tonen, filteren, sorteren.

Belangrijke concepten:

- realtime listener met onSnapshot op matches
- auth-aware listener management met onAuthStateChanged
- verlopen wedstrijden worden verwijderd
- useMemo voor filtering/sorting

Filter states:

- status, format, type, level, sortKey

Waarom unsubscribe refs:

- om listeners te stoppen bij logout en permission-denied te vermijden.

Docent-vraag:

- "Waarom useMemo hier?"

Antwoord:

- "Omdat gefilterde lijst afgeleid is van state en duurder kan zijn; useMemo voorkomt onnodige recalculaties op elke render."

app/(tabs)/create.tsx (Wedstrijd aanmaken)

Doel:

- wizard in 4 stappen om match te publiceren.

Stappen:

1. club kiezen
2. datum kiezen
3. tijdslot kiezen
4. details instellen

Belangrijke logic:

- haalt clubs en appConfig uit Firestore
- checkt bezette slots in bookings en matches
- custom range slider met PanResponder voor min/max niveau
- schrijft nieuw match document

Belangrijke velden in nieuwe match:

- date, time, club, clubId
- minLevel, maxLevel
- format, maxPlayers
- isMixed, isCompetitive
- players [creator uid]
- createdBy, createdAt, status

Docent-vraag:

- "Waarom check op dubbele slot in twee collecties?"

Antwoord:

- "Een slot kan bezet zijn door losse veldboeking of door bestaande match; beide moeten conflictvrij blijven."

app/(tabs)/book.tsx (Veld boeken)

Doel:

- losse veldreservering (geen match) in 3 stappen.

Stappen:

1. club
2. datum
3. tijdslot

Belangrijke logic:

- leest clubs + timeSlots config
- vraagt al geboekte slots op uit bookings
- blokkeert verleden slots met isSlotPast
- schrijft booking document met user info

Resultaat:

- simulatie van betaling in Alert.

app/(tabs)/profile.tsx (Profiel)

Doel:

- gebruiker info, eigen matches, eigen bookings, logout.

Belangrijke logic:

- onAuthStateChanged voor veilige auth state sync
- users/{uid} level ophalen of initialiseren op 1.5
- realtime listeners op:
 - matches where createdBy == uid
 - bookings where userId == uid
- listeners expliciet stoppen voor signOut

Waarom belangrijk:

- voorkomt Firestore permission errors bij uitloggen.

app/match/[id].tsx (Match detail + chat)

Doel:

- detailpagina per match + interacties.

Functionaliteit:

- realtime match data
- realtime chat subcollection messages
- inschrijven / uitschrijven
- verwijderen (enkel maker)
- resultaat invoeren (competitieve, afgelopen wedstrijd)
- level updates op basis van resultaat

Belangrijke businessregels:

- join niet als vol of al voorbij
- creator kan zichzelf niet "leave" doen, wel delete
- status update naar vol/open bij join/leave
- resultaat:
 - winnaars +0.2 (max 7.0)
 - verliezers -0.1 (min 0.5)
 - draw geen wijziging

Chat model:

- match document heeft subcollection messages
- system berichten en user berichten

Docent-vraag:

- "Waarom match chat in subcollection?"

Antwoord:

- "Dan blijven berichten logisch onder 1 match gegroepeerd en kunnen we realtime, ordered queries per match doen."

5. Data model in Firestore

Collectie: users

Document id: uid

Velden (gebruikt in code):

- name
- email
- level
- uid

Collectie: clubs

Document id: club id (string)

Velden:

- name
- address

Collectie: appConfig

Document id: settings

Velden:

- timeSlots: string[]
- formats: string[]
- levelMin, levelMax, levelStep

Collectie: bookings

Velden:

- clubId, clubName
- date, time
- userId, userEmail
- createdAt

Collectie: matches

Velden:

- date, time
- club, clubId
- minLevel, maxLevel
- format, maxPlayers
- isMixed, isCompetitive
- players: uid[]
- createdBy, createdAt
- status: openIvol
- result (optioneel): { scoreA, scoreB, winner }

Subcollectie per match:

- messages
 - text, userId/userEmail of system flag
 - createdAt

6. React basis uitgelegd met voorbeelden uit jullie app

useState

Wat is het:

- lokale component state.

Voorbeeld:

- in login: email/password/isLogin/loading.

Waarom:

- UI reageert automatisch op state veranderingen.

useEffect

Wat is het:

- side effects uitvoeren na render.

Voorbeeld:

- config laden vanuit Firestore bij openen van create/book.
- listeners starten en cleanup doen.

useMemo

Wat is het:

- afgeleide data cachen tussen renders.

Voorbeeld:

- filteredMatches in matches.tsx.

useRef

Wat is het:

- mutable referentie zonder rerender.

Voorbeeld:

- unsubscribe functies bewaren.

- scrollViewRef gebruiken voor scrollToEnd.

Conditional rendering

Wat is het:

- verschillende UI tonen op basis van state.

Voorbeeld:

- create/book stap 1/2/3/4.
- detailpagina toont andere actiekноп afhankelijk van isJoined/isCreator/isPast.

7. Navigatieconcept (Expo Router)

Belangrijk:

- map-structuur = routes.

Voorbeelden:

- app/index.tsx -> /
- app/(tabs)/matches.tsx -> /(tabs)/matches
- app/match/[id].tsx -> /match/:id

Dynamic route:

- [id] betekent parameter.
- useLocalSearchParams leest deze uit.

8. Auth + Firestore samenwerking

Flow register:

1. Firebase Auth maakt user account.
2. Firestore users/{uid} krijgt profielvelden.

Flow login:

1. Auth signin
2. router naar tabs
3. schermen luisteren op user-afhankelijke data

Belangrijk punt:

- auth.currentUser alleen gebruiken als momentopname.
- voor betrouwbare updates: onAuthStateChanged.

9. Veiligheid en best practices (wat goed is, wat beter kan)

Goed in huidige code:

- listeners worden meestal correct opgeruimd.
- checks op bezette slots.
- status updates op match capaciteit.

Verbeteringen die je kan noemen:

- transacties gebruiken voor join/leave om race conditions te vermijden.
- serverTimestamp() gebruiken i.p.v. client Date() voor consistente tijd.
- Firestore rules harden op ownership en velden.
- TypeScript types/interfaces toevoegen i.p.v. veel any.
- API keys in client zijn normaal publiek bij Firebase, maar rules moeten strikt zijn.

10. Veelgestelde docentvragen + modelantwoorden

1. "Waarom Expo Router in plaats van handmatige React Navigation setup?"
 - "Omdat bestandsgebaseerde routing eenvoudiger onderhoudbaar is; route-definitie zit direct in mappenstructuur."
2. "Waarom onSnapshot en niet getDocs overal?"
 - "onSnapshot geeft realtime updates zonder handmatige refresh. Voor lijsten zoa

ls matches/chat is dat essentieel."

3. "Wat gebeurt er als gebruiker uitlogt terwijl listener actief is?"

- "Dan kunnen permission-denied errors ontstaan; daarom stoppen we listeners expliciet op auth change en cleanup."

4. "Hoe voorkom je dubbele boeking?"

- "Bij create controleren we beide collecties (bookings + matches) op zelfde club/datum/tijd voor insert."

5. "Waar wordt business logic van niveau-updates gedaan?"

- "In match detail bij resultaatinvoer; win +0.2, verlies -0.1, met min/max clamps."

11. Praktische uitlegflow (als jij het live moet presenteren)

Aanpak in 8 minuten:

1. Start met architectuur (Expo Router + Firebase).
2. Toon app/_layout.tsx en tab layout.
3. Toon login flow en users doc creation.
4. Toon create flow (4 stappen + slot conflict check).
5. Toon matches realtime + filters.
6. Toon match detail (join/leave/chat/result logic).
7. Toon profile listeners + veilige logout.
8. Eindig met Firestore schema en verbeterpunten.

12. Commando's die je moet kennen

- npm install
- npm run start
- npm run android
- npm run ios
- npm run web
- npm run seed:firebase

13. Snelle samenvatting in 1 minuut

"Dit is een Expo React Native app met file-based routing via Expo Router. Authenticatie loopt via Firebase Auth en alle domeindata zit in Firestore. De hoofdfunctionaliteit is wedstrijden beheren, veld boeken en realtime chatten per wedstrijd. De app gebruikt listeners (onSnapshot) voor realtime updates en onAuthStateChanged voor veilige listener lifecycle bij login/logout. De belangrijkste data-entiteiten zijn users, clubs, appConfig, bookings en matches met messages als subcollectie."

Als je wilt, kan ik hierna ook een tweede versie maken met:

- diagrammen (route flow + data flow)
- examenvragen met korte antwoorden
- een 15-min spreekscript dat je letterlijk kan oefenen.